AWS SERIES · MODULE 2 OF 8

AWS IAM for Beginners: Who Can Do What

IAM decides who may do what in your AWS account. This guide explains users, groups, roles and policies in plain words, shows how to read a policy line by line, and ends with the exact policy and role the Lambda lab uses.

PA

Tutor: Prudhvi Akella
Lead Data & AI Engineer · 13+ years in data engineering and AI architecture

Services: AWS Identity and Access Management (IAM) · AWS STS (temporary keys, used automatically)

Before this module: an AWS account. Next: the lab "Lambda loads a file from S3 into DynamoDB", which uses everything here. Examples use account `123456789012` and Region `us-east-1`

1. The one question IAM answers
2. Who can sign in: root user and IAM users
3. Users, groups and roles
4. A policy, line by line
5. Actions: what can be done
6. ARNs: the full name of a resource
7. How AWS decides: allow or deny
8. AWS managed policies and your own
9. Roles: permissions for AWS services
10. Example: the policy for our Lambda lab
11. Least privilege: give only what is needed
12. Reading an AccessDenied error
13. Try it yourself (read only)
14. Good habits
15. What you can skip for now
16. Words to know
17. Key takeaways

1 The one question IAM answers

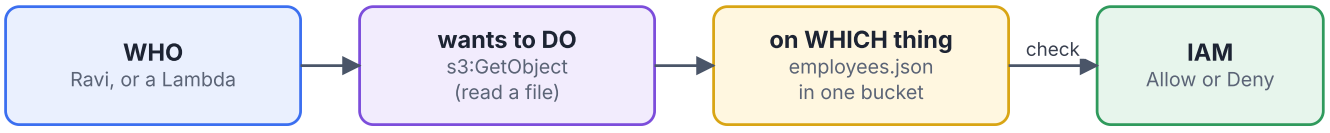


Figure 1. Every request to AWS is checked by IAM. It answers one question: is this allowed?

Every time anyone uses AWS — clicking in the console, typing a CLI command, or code running in a Lambda — AWS asks IAM: **can this WHO do this ACTION on this RESOURCE?** IAM says Allow or Deny. That is all IAM does.

Part	Means	Example
Who (principal)	The person or thing making the request	the user `ravi`, or the Lambda `s3-to-dynamodb`

Part	Means	Example
Action	What they want to do	`s3:GetObject` — read a file from S3
Resource	Which exact thing	the file `employees.json` in bucket `vs-lab-employees-123456789012`

IAM is **global**: it is not tied to a Region. A user or role you create can be used in every Region.

2 Who can sign in: root user and IAM users

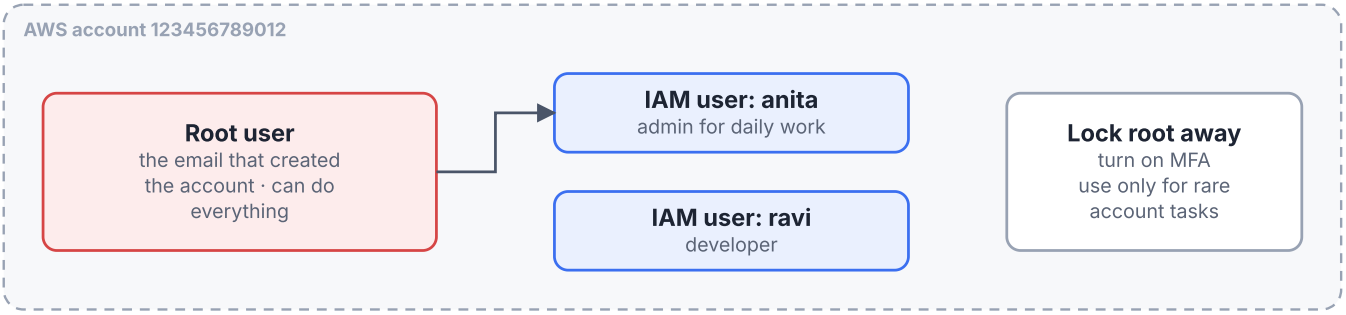


Figure 2. The root user owns the account. Everyone else gets their own IAM user (or role) with only the rights they need.

	Root user	IAM user
Who	The email address used to open the account	A person (or tool) you create inside the account
What it can do	Everything, and IAM cannot limit it	Only what its policies allow
Daily use?	**No.** Turn on MFA and put it away	Yes — every person gets their own
Sign-in	Email + password (+ MFA)	Account ID or alias + user name + password (+ MFA)

MFA (multi-factor authentication) means a second check after the password — a code from an app on your phone. Turn it on for root on day one.

In companies people usually sign in through **IAM Identity Center** (single sign-on) instead of IAM users. The idea is the same: each person gets only the permissions they need.

3 Users, groups and roles

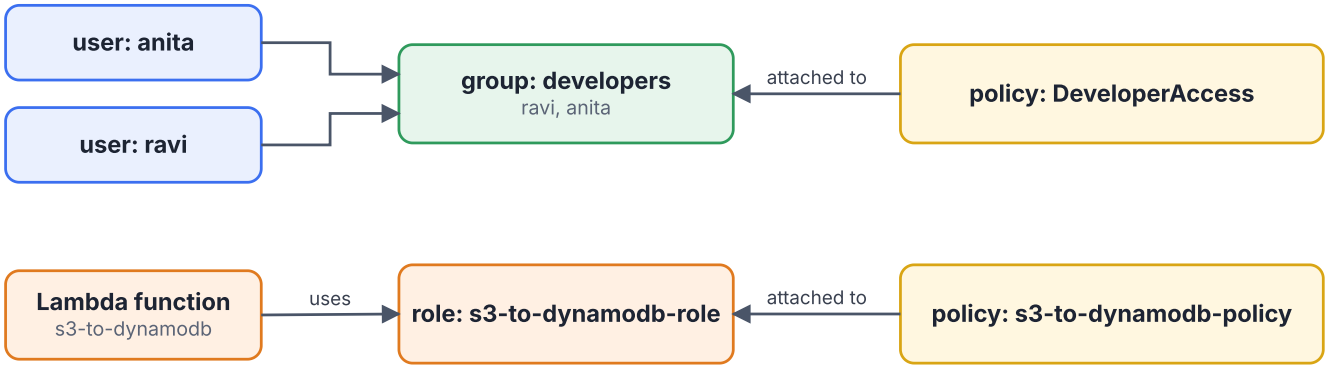


Figure 3. People get users (in groups). AWS services and apps get roles. Policies attach to all three.

Thing	What it is	Use it for
User	One identity with its own sign-in	One person
Group	A set of users. Attach a policy once, every member gets it	A team: developers, HR, finance
Role	A set of permissions that is **used for a short time** by someone or something trusted. No password	AWS services (Lambda, EC2) and apps
Policy	A JSON document that lists what is allowed	Attach to a user, group or role

> Rule of thumb: **people → users (or SSO); AWS services → roles.** A Lambda never gets a user or a password — it gets a role.

4 A policy, line by line

A **policy** is a small JSON document (Day 04 of the Python course). This one lets its owner read files from one bucket:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadEmployeeFiles",
      "Effect": "Allow",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::vs-lab-employees-123456789012/*"
    }
  ]
}
```

Line	Means	In this policy
`Version`	The policy language version. Always write `2012-10-17` (it is not today's date)	—

Line	Means	In this policy
`Statement`	A list of rules. A policy can have one or many	one rule
`Sid`	Optional label for the rule, for humans	"ReadEmployeeFiles"
`Effect`	`Allow` or `Deny`	Allow
`Action`	What may be done (next section)	read a file from S3
`Resource`	Which exact things (the section after)	every file in that one bucket

Read a statement as one sentence: **Allow** reading files (`s3:GetObject`) for **every file** (`/*`) in the bucket `vs-lab-employees-123456789012` .

5 Actions: what can be done

An action is written `service:Operation`. Each AWS service has its own list. These are the ones you meet first:

Action	Allows
`s3:GetObject`	Read (download) a file from S3
`s3:PutObject`	Upload a file to S3
`s3:ListBucket`	See the list of files in a bucket
`dynamodb:PutItem`	Add (or replace) one item in a table
`dynamodb:GetItem`	Read one item by its key
`logs:PutLogEvents`	Write log lines to CloudWatch Logs

`*` means "everything": `s3:*` is every S3 action, and `"Action": "*"` is every action in AWS. Beginners use `*` to make an error go away — and give far more than needed. Name the actions instead.

6 ARNs: the full name of a resource

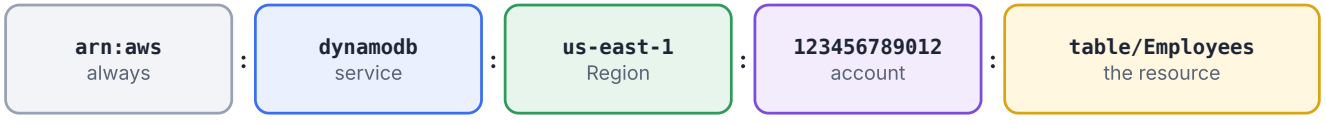


Figure 4. An ARN (Amazon Resource Name) is the full address of one resource.

The `Resource` line needs the exact name of the thing. AWS calls it an **ARN**. The parts are joined with `:`.

Resource	ARN
The DynamoDB table `Employees`	`arn:aws:dynamodb:us-east-1:123456789012:table/Employees`
The bucket itself	`arn:aws:s3:::vs-lab-employees-123456789012`
Every file in the bucket	`arn:aws:s3:::vs-lab-employees-123456789012/*`
An IAM role	`arn:aws:iam::123456789012:role/s3-to-dynamodb-role`
Every resource	`*`

S3 ARNs have **no Region and no account** (`arn:aws:s3:::name`), because bucket names are already unique worldwide. IAM ARNs have **no Region** (`arn:aws:iam::123…`), because IAM is global.

File actions like `s3:GetObject` need the files: `bucket-name/*` . Writing only `bucket-name` (without `/*`) is a very common cause of AccessDenied.

7 How AWS decides: allow or deny

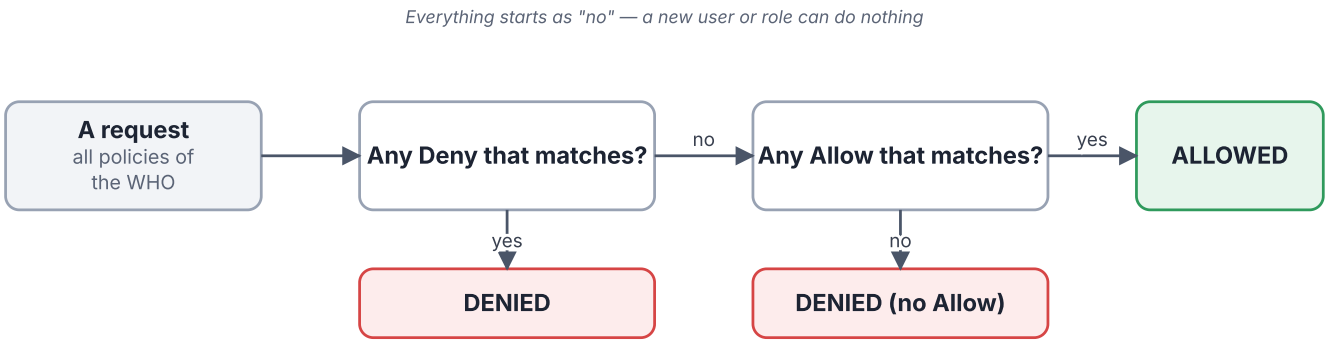


Figure 5. Start at No. An Allow opens the door. An explicit Deny always closes it.

1. **Start at no.** A brand-new user or role can't do anything.
2. **Look for a Deny.** If any policy says Deny for this action and resource, the answer is no. Nothing can override it.
3. **Look for an Allow.** If a policy says Allow, the answer is yes.
4. **Nothing matched?** The answer is no. You will see `AccessDenied` .

Remember: **Deny beats Allow, and silence means No.** In this course we only write Allow rules — everything we don't allow is already denied.

8 AWS managed policies and your own

	AWS managed policy	Customer managed policy (your own)
Written by	AWS	You

	AWS managed policy	Customer managed policy (your own)
Example	`AmazonS3ReadOnlyAccess` — read **every** bucket in the account	`s3-to-dynamodb-policy` — read **one** bucket, write **one** table
Changes	AWS updates it	Only when you edit it
Good for	Quick starts, broad job roles	Apps and Lambdas: exactly what they need

Both kinds are created once and can be **attached** to many users, groups and roles. In the IAM console, **Policies** lists both — the **Type** column says "AWS managed" or "Customer managed".

> In the lab you create your own policy, so you can see and understand every permission the Lambda has.

9 Roles: permissions for AWS services

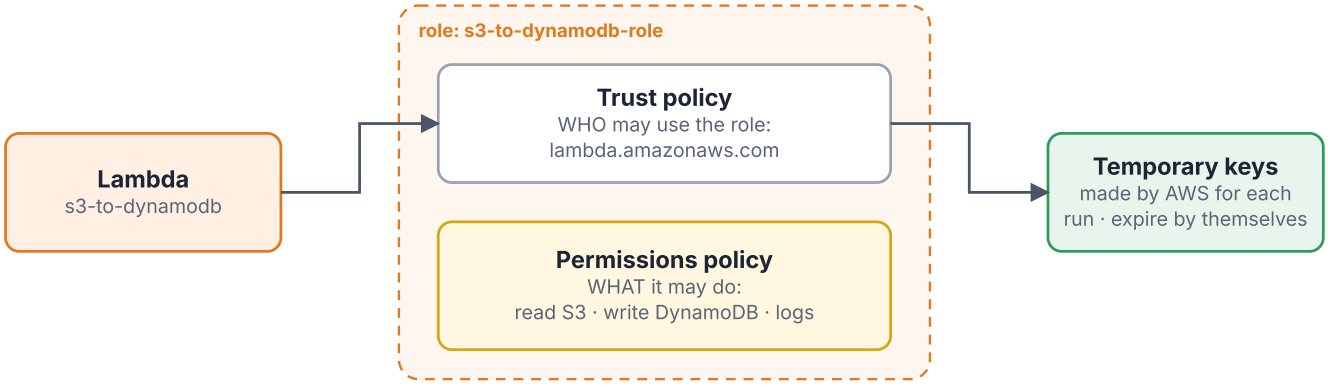


Figure 6. A role has two parts: who may use it (trust policy) and what it may do (permissions policy).

Your Lambda code calls S3 and DynamoDB, so it needs permissions. It must not have a password or access keys written into the code. Instead you give the Lambda a **role** (its *execution role*).

1. **The trust policy says who may use the role.** For a Lambda role: the Lambda service. The console writes this for you when you pick "Lambda" as the use case.
2. **The permissions policy says what the role may do.** Read the file, write the table, write logs.
3. **AWS hands out temporary keys.** Each time the function runs, AWS gives it short-lived keys for the role. You write no key in the code — boto3 finds them by itself.

```
// Trust policy of s3-to-dynamodb-role
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": { "Service": "lambda.amazonaws.com" },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

`sts:AssumeRole` means "take on this role". `Principal` is the WHO that is trusted — here the Lambda service. This is the only policy in this course that has a Principal line.

Never put access keys in code or in Lambda settings. If the code leaks, the keys leak. A role has no keys to leak.

10 Example: the policy for our Lambda lab

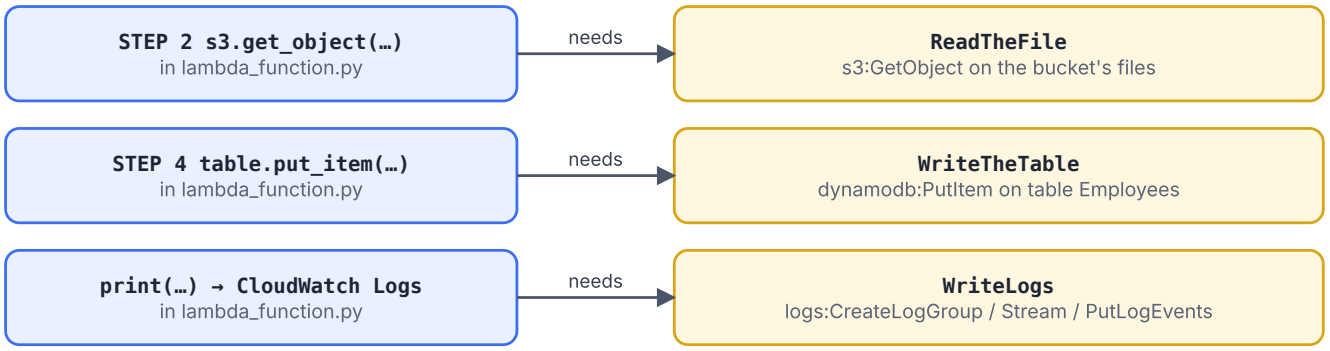


Figure 7. Each thing the Lambda code does needs one statement in its policy.

In the lab, a Lambda reads `employees.json` from S3 and saves each employee in the DynamoDB table `Employees`. This is its whole permissions policy — three statements, nothing else:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadTheFile",
      "Effect": "Allow",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::vs-lab-employees-123456789012/*"
    },
    {
      "Sid": "WriteTheTable",
      "Effect": "Allow",
      "Action": "dynamodb:PutItem",
      "Resource": "arn:aws:dynamodb:us-east-1:123456789012:table/Employees"
    },
    {
      "Sid": "WriteLogs",
      "Effect": "Allow",
      "Action": ["logs:CreateLogGroup", "logs:CreateLogStream", "logs:PutLogEvents"],
      "Resource": "*"
    }
  ]
}
```

Statement	Allows	Only on
`ReadTheFile`	Reading files	files in the lab bucket (`/*`)
`WriteTheTable`	Saving items	the table `Employees` in `us-east-1`

Statement	Allows	Only on
`WriteLogs`	Writing log lines	any log group (`*`) — standard for Lambda logs

What it can **not** do: delete files, read other buckets, read or delete items, touch other tables, create users. Anything not listed is denied.

11 Least privilege: give only what is needed

Least privilege means each user or role gets only the permissions it needs, on only the resources it needs.

Too wide (avoid)	Just right (lab)	Why it matters
`"Action": "s3:*"`	`"Action": "s3:GetObject"`	The Lambda only reads; with `s3:*` a bug could delete files
`"Resource": "*"` for S3	`arn:aws:s3:::vs-lab-employees-…/*`	Other buckets stay out of reach
`AmazonDynamoDBFullAccess`	`dynamodb:PutItem` on `table/Employees`	It cannot delete tables or read customer data

Start small. If something fails with AccessDenied, add the **one** missing action — not `*` .

12 Reading an AccessDenied error

When IAM says no, the error tells you all three parts of the question. An error like this is what you see if the Lambda's policy is missing `dynamodb:PutItem` :

```
An error occurred (AccessDeniedException) when calling the PutItem operation:
User: arn:aws:sts::123456789012:assumed-role/s3-to-dynamodb-role/s3-to-dynamodb
is not authorized to perform: dynamodb:PutItem
on resource: arn:aws:dynamodb:us-east-1:123456789012:table/Employees
because no identity-based policy allows the dynamodb:PutItem action
```

Part of the message	Tells you	Here
`User: …assumed-role/<role>/<session>`	WHO — the role (and which function used it)	`s3-to-dynamodb-role`
`not authorized to perform: …`	ACTION that was refused	`dynamodb:PutItem`
`on resource: …`	RESOURCE	the `Employees` table
`because no identity-based policy allows …`	WHY	no Allow for it — add it to the role's policy

Fix checklist: (1) is the policy attached to the **right role**? (2) is the **action** spelt the same? (3) does the **resource ARN** match exactly — Region, account ID, table or bucket name, `/*` ?

13 Try it yourself (read only)

Nothing here changes your account.

Console

1. Open **IAM** → **Policies**. Search `AmazonS3ReadOnlyAccess` and open it.
2. Click the **JSON** button (or tab): find `Effect`, `Action` and `Resource`. Notice `"Resource": "*"` — every bucket.
3. Open **IAM** → **Roles**. Any role whose name starts with `AWSServiceRoleFor…` was made by AWS for a service. Open one and look at its **Trust relationships** tab: the `Principal` is an AWS service.

Command line — Mac and Windows (same command)

Ask AWS "who am I?" — the WHO part of every request you send:

```
aws sts get-caller-identity
```

```
{
    "UserId": "AIDAXXXXXXXXXXXXXXXXX",
    "Account": "123456789012",
    "Arn": "arn:aws:iam::123456789012:user/ravi"
}
```

> Inside the Lambda, the same call would show `arn:aws:sts::123456789012:assumed-role/s3-to-dynamodb-role/s3-to-dynamodb` — the function is using its role.

14 Good habits

Do	Why
Turn on MFA for the root user, then stop using it	Root can do everything and IAM cannot limit it
One user (or SSO login) per person — never share	You can see who did what, and remove one person easily
Give AWS services a role, never access keys	Roles use temporary keys; nothing to leak
Name actions and resources; avoid `*`	Least privilege limits the damage of a mistake
Put permissions on groups, not single users	When people join or leave, change the group only

15 What you can skip for now

Topic	What it is, in one line	When you need it
Conditions	Extra rules in a policy: only from this IP, only with MFA, only with this tag	Tighter security rules
Resource-based policies	A policy on the resource itself (e.g. a bucket policy) that says who may use it	Sharing a bucket with another account or service

Topic	What it is, in one line	When you need it
Cross-account access	A role in one account used from another account	Companies with many AWS accounts
Permission boundaries	An upper limit on what one user or role can ever get	Letting teams create their own roles safely
SCPs (AWS Organizations)	An upper limit for a whole account	Central admins of many accounts
IAM Identity Center setup	Single sign-on for people	When an organisation stops using IAM users

16 Words to know

Word	Meaning
Principal	The WHO of a request: a user, a role, or an AWS service
Policy	JSON document of Allow (or Deny) rules
Statement	One rule inside a policy: Effect + Action + Resource
ARN	Full name of a resource, e.g. `arn:aws:dynamodb:us-east-1:123456789012:table/Employees`
Role	Permissions used for a short time by someone trusted; no password
Execution role	The role a Lambda function uses when it runs
Trust policy	The part of a role that says who may use it
Least privilege	Only the actions and resources that are needed
MFA	A second check after the password (a code on your phone)

17 Key takeaways

1. IAM answers one question: can this WHO do this ACTION on this RESOURCE?
2. Root user: lock it with MFA and don't use it daily. People get users; AWS services get roles.
3. A policy statement = Effect + Action + Resource. Read it as one sentence.
4. Everything starts as No. An Allow opens it; a Deny always wins.
5. A role = trust policy (who may use it) + permissions policy (what it may do). AWS gives it temporary keys.
6. Give only what is needed — name the actions and the exact ARNs.
7. An AccessDenied message tells you the who, the action and the resource — fix exactly that.